

09 - Evoluzione del software

Francesco Betti Sorbelli

Università degli studi di Perugia. `francesco.bettisorbelli@unipg.it`



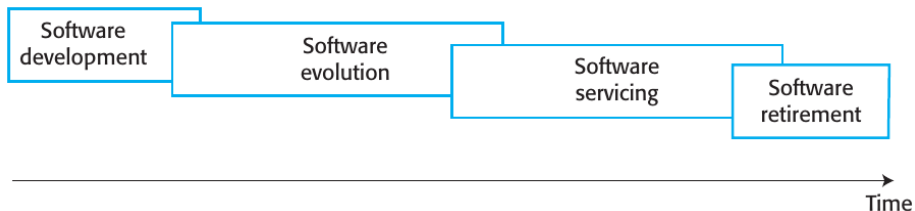
Argomenti trattati

1 Processi di evoluzione

2 Sistemi legacy

3 Manutenzione del SW

- Il cambiamento del SW è inevitabile
 - Con l'utilizzo del SW emergono nuovi requisiti;
 - L'ambiente commerciale cambia;
 - Gli errori devono essere fixati;
 - Vengono aggiunti al sistema nuovi computer e apparecchiature;
 - Potrebbe essere necessario migliorare le prestazioni o l'affidabilità del sistema.
- Un problema fondamentale per tutte le organizzazioni è l'implementazione e la gestione delle modifiche ai sistemi SW esistenti.
- Le organizzazioni investono moltissimo nei loro sistemi SW: sono risorse aziendali critiche.
- La maggior parte del budget per il SW nelle grandi aziende è dedicato alla modifica e all'evoluzione del SW esistente piuttosto che allo sviluppo di nuovo SW.



- L'evoluzione

- La fase del ciclo di vita di un sistema SW in cui è in uso operativo e si evolve man mano che nuovi requisiti vengono proposti e implementati nel sistema.

- Assistenza

- In questa fase, il SW rimane utile, ma le uniche modifiche apportate sono quelle necessarie per mantenerlo operativo, cioè la correzione di bug e le modifiche per riflettere i cambiamenti nell'ambiente del SW. Non vengono aggiunte nuove funzionalità.

- Eliminazione graduale

- Il SW può ancora essere utilizzato, ma non viene più modificato.

Argomenti trattati

1 Processi di evoluzione

2 Sistemi legacy

3 Manutenzione del SW

Processi evolutivi

- I processi di evoluzione del SW dipendono da
 - Il tipo di SW sottoposto a manutenzione;
 - I processi di sviluppo utilizzati;
 - Le competenze e l'esperienza delle persone coinvolte.
- Le proposte di cambiamento sono il motore dell'evoluzione del sistema.
 - Dovrebbe essere collegato ai componenti interessati dalla modifica, consentendo così di stimare il costo e l'impatto della modifica.
- L'identificazione e l'evoluzione dei cambiamenti continuano per tutta la durata del sistema.

Implementazione del cambiamento

- Iterazione del processo di sviluppo in cui le revisioni del sistema vengono progettate, implementate e testate.
- Una differenza critica è che la prima fase dell'implementazione del cambiamento può coinvolgere la comprensione del programma, soprattutto se gli sviluppatori del sistema originale non sono responsabili dell'implementazione del cambiamento.
- Durante la fase di comprensione del programma, bisogna capire come è strutturato il programma, come fornisce le funzionalità e come la modifica proposta potrebbe influire sul programma.

Richieste di modifica urgenti

- Le modifiche urgenti possono essere implementate senza passare attraverso tutte le fasi del processo di ingegneria del SW.
 - Se è necessario riparare un guasto grave del sistema per consentire il normale funzionamento;
 - Se le modifiche all'ambiente del sistema (ad esempio, un aggiornamento del sistema operativo) hanno effetti imprevisti;
 - Se ci sono cambiamenti aziendali che richiedono una risposta molto rapida (ad esempio, il rilascio di un prodotto concorrente).

Metodi agili ed evoluzione

- I metodi agili si basano sullo sviluppo incrementale, per cui il passaggio dallo sviluppo all'evoluzione avviene senza soluzione di continuità (**seamless**).
 - L'evoluzione è semplicemente la continuazione del processo di sviluppo basato su frequenti rilasci del sistema.
- I test di regressione automatizzati sono particolarmente utili quando vengono apportate modifiche a un sistema.
- Le modifiche possono essere espresse come storie utente aggiuntive.

Problemi di passaggio di consegne

- Team di sviluppo vs team di evoluzione.
- Quando il team di sviluppo ha utilizzato un approccio agile, ma il team di evoluzione non ha familiarità con i metodi agili e preferisce un approccio pianificato.
 - Il team di evoluzione potrebbe aspettarsi una documentazione dettagliata a supporto dell'evoluzione, che non viene prodotta nei processi agili.
- Quando per lo sviluppo è stato utilizzato un approccio pianificato, ma il team di evoluzione preferisce utilizzare metodi agili.
 - Il team di evoluzione potrebbe dover ricominciare da zero a sviluppare test automatizzati e il codice del sistema potrebbe non essere stato rifattorizzato e semplificato come ci si aspetta nello sviluppo agile.

Argomenti trattati

1 Processi di evoluzione

2 Sistemi legacy

3 Manutenzione del SW

Sistemi legacy

- I sistemi legacy sono sistemi vecchi che si basano su linguaggi e tecnologie non più utilizzati per lo sviluppo di nuovi sistemi.
- Il SW legacy può dipendere da HW datato, come i computer mainframe, e può avere processi e procedure legacy associati.
- I sistemi legacy non sono solo sistemi SW, ma sistemi socio-tecnici più ampi che comprendono HW, SW, librerie e altri SW di supporto e processi aziendali.
- HW del sistema: I sistemi legacy possono essere stati scritti per un HW non più disponibile.
- SW di supporto: Il sistema legacy può fare affidamento su una serie di SW di supporto, che possono essere obsoleti o non supportati.

Sostituzione del sistema legacy

- La sostituzione dei sistemi legacy è rischiosa e costosa, quindi le aziende continuano a utilizzare questi sistemi.
- La sostituzione del sistema è rischiosa per una serie di motivi
 - Mancanza di specifiche complete del sistema
 - Stretta integrazione tra sistema e processi aziendali
 - Regole di business non documentate incorporate nel sistema legacy
 - Lo sviluppo di nuovo SW può essere in ritardo e/o fuori budget

Modifica del sistema legacy

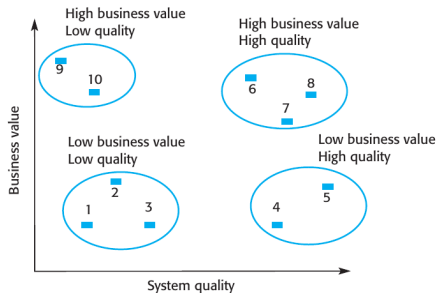
- I sistemi legacy sono costosi da modificare per una serie di motivi:
 - Nessuno stile di programmazione coerente
 - Utilizzo di linguaggi di programmazione obsoleti, con poche persone disponibili con queste competenze linguistiche.
 - Documentazione di sistema inadeguata
 - Degrado della struttura del sistema
 - Le ottimizzazioni dei programmi possono renderli di difficile comprensione
 - Errori, duplicazioni e incoerenze dei dati

Gestione dei sistemi legacy

- Le organizzazioni che si affidano a sistemi legacy devono scegliere una strategia per far evolvere questi sistemi.
 - Eliminare completamente il sistema e modificare i processi aziendali in modo che non sia più necessario;
 - Continuare a mantenere il sistema;
 - Trasformare il sistema attraverso la reingegnerizzazione per migliorarne la manutenibilità;
 - Sostituire il sistema con uno nuovo.
- La strategia scelta deve dipendere dalla qualità del sistema e dal suo valore commerciale.

Categorie di sistemi legacy

- **Bassa qualità, basso valore commerciale**
 - Questi sistemi dovrebbero essere eliminati.
- **Bassa qualità, alto valore commerciale**
 - Questi sistemi forniscono un importante contributo all'azienda, ma sono costosi da mantenere. Dovrebbero essere riprogettati o sostituiti se è disponibile un sistema adeguato.
- **Alta qualità, basso valore commerciale**
 - Sostituire con COTS, rottamare completamente o mantenere.
- **Alta qualità, alto valore commerciale**
 - Continuare a funzionare con la normale manutenzione del sistema.



Valutazione del valore aziendale

- La valutazione deve tenere conto dei diversi punti di vista
 - Utenti finali del sistema;
 - Clienti commerciali;
 - Manager di linea;
 - Manager IT;
 - Dirigenti di alto livello.
- Chiederei feedback/pareri dalle diverse parti interessate e raccogliere i risultati.

Problemi di valutazione del valore aziendale

- Basso vs alto valore aziendale?
- L'uso del sistema
 - Se i sistemi sono utilizzati solo occasionalmente o da un numero ridotto di persone, possono avere un basso valore aziendale.
- I processi aziendali supportati
 - Un sistema può avere un basso valore aziendale se costringe all'uso di processi aziendali inefficienti.
- Affidabilità del sistema
 - Se un sistema non è affidabile e i problemi si ripercuotono direttamente sui clienti dell'azienda, il sistema ha un basso valore commerciale.
- Le uscite del sistema
 - Se l'azienda dipende dai risultati del sistema, allora il sistema ha un elevato valore aziendale.

Valutazione della qualità del sistema

- Valutazione dei processi aziendali
 - In che misura il processo aziendale supporta gli obiettivi attuali dell'azienda?
- Valutazione ambientale
 - Quanto è efficace l'ambiente del sistema e quanto è costosa la sua manutenzione?
- Valutazione dell'applicazione
 - Qual è la qualità del sistema SW applicativo?
- ⇒ È bene porsi delle domande...

Fattori utilizzati nella valutazione dell'ambiente

Fattore	Domande
Stabilità dei fornitori	Il fornitore esiste ancora? Il fornitore è finanziariamente stabile e probabilmente continuerà ad esistere? Se il fornitore non è più in attività, qualcun altro si occupa della manutenzione dei sistemi?
Tasso di fallimento	L'HW ha un'alta percentuale di guasti segnalati? Il SW di supporto si blocca e costringe a riavviare il sistema?
Età	Quanto sono vecchi l'HW e il SW? Quanto più vecchi sono l'HW e il SW di supporto, tanto più obsoleti saranno. Può ancora funzionare correttamente, ma il passaggio a un sistema più moderno potrebbe comportare notevoli vantaggi economici e commerciali.
Prestazioni	Le prestazioni del sistema sono adeguate? I problemi di prestazioni hanno un effetto significativo sugli utenti del sistema?

Fattori utilizzati nella valutazione dell'ambiente

Fattore	Domande
Requisiti di supporto	Qual è il supporto locale richiesto dall'HW e dal SW? Se i costi associati a questa assistenza sono elevati, può valere la pena di considerare la sostituzione del sistema.
Costi di manutenzione	Quali sono i costi di manutenzione dell'HW e delle licenze SW di supporto? L'HW più vecchio può avere costi di manutenzione più elevati rispetto ai sistemi moderni. Il SW di supporto può avere costi di licenza annuali elevati.
Interoperabilità	Ci sono problemi di interfacciamento del sistema con altri sistemi? I compilatori, ad esempio, possono essere utilizzati con le versioni attuali del sistema operativo? È necessaria l'emulazione HW?

Fattori utilizzati nella valutazione delle applicazioni

Fattore	Domande
Comprensibilità	Quanto è difficile comprendere il codice sorgente del sistema attuale? Quanto sono complesse le strutture di controllo utilizzate? Le variabili hanno nomi significativi che riflettono la loro funzione?
Documentazione	Quale documentazione del sistema è disponibile? La documentazione è completa, coerente e aggiornata?
Dati	Esiste un modello di dati esplicito per il sistema? In che misura i dati sono duplicati nei vari file? I dati utilizzati dal sistema sono aggiornati e coerenti?
Prestazioni	Le prestazioni dell'applicazione sono adeguate? I problemi di prestazioni hanno un effetto significativo sugli utenti del sistema?

Fattori utilizzati nella valutazione delle applicazioni

Fattore	Domande
Linguaggio di programmazione	Sono disponibili compilatori moderni per il linguaggio di programmazione utilizzato per sviluppare il sistema? Il linguaggio di programmazione è ancora utilizzato per lo sviluppo di nuovi sistemi?
Gestione della configurazione	Tutte le versioni di tutte le parti del sistema sono gestite da un sistema di gestione della configurazione? Esiste una descrizione esplicita delle versioni dei componenti utilizzati nel sistema attuale?
Dati del test	Esistono dati di test per il sistema? Esiste un registro dei test di regressione eseguiti quando sono state aggiunte nuove funzionalità al sistema?
Competenze del personale	Ci sono persone disponibili che hanno le competenze per mantenere l'applicazione? Ci sono persone disponibili che hanno esperienza con il sistema?

Misurazione del sistema

- È possibile raccogliere dati quantitativi per valutare la qualità del sistema applicativo.
 - Il numero di richieste di modifica del sistema; più alto è questo valore accumulato, più bassa è la qualità del sistema.
 - Il numero di interfacce utente diverse utilizzate dal sistema; più sono le interfacce, più è probabile che ci siano incongruenze e ridondanze in queste interfacce.
 - Il volume dei dati utilizzati dal sistema. Con l'aumento del volume di dati (numero di file, dimensioni del database, ecc.) elaborati dal sistema, aumentano anche le incongruenze e gli errori in tali dati.
 - La pulizia dei vecchi dati è un processo molto costoso e dispendioso in termini di tempo.

Argomenti trattati

1 Processi di evoluzione

2 Sistemi legacy

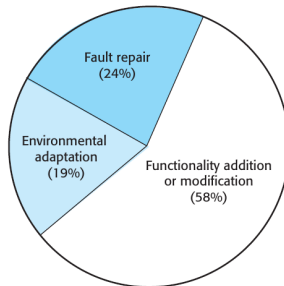
3 Manutenzione del SW

Manutenzione del SW

- Modificare un programma dopo che è stato messo in uso.
- Il termine è usato soprattutto per modificare il SW personalizzato. Si dice che i prodotti SW generici si evolvono per creare nuove versioni.
- La manutenzione non comporta normalmente modifiche sostanziali all'architettura del sistema.
- I cambiamenti vengono implementati modificando i componenti esistenti e aggiungendo nuovi componenti al sistema.

Tipi di manutenzione

- Riparazione dei guasti
 - Modificare un sistema per risolvere bug e correggere le carenze nel modo in cui soddisfa i requisiti.
- Adattamento dell'ambiente
 - Manutenzione per adattare il SW a un ambiente operativo diverso
 - Modifica di un sistema in modo che operi in un ambiente diverso (computer, sistema operativo, ecc.) rispetto alla sua implementazione iniziale.
- Aggiunta e modifica di funzionalità
 - Modificare il sistema per soddisfare nuovi requisiti.



Costi di manutenzione

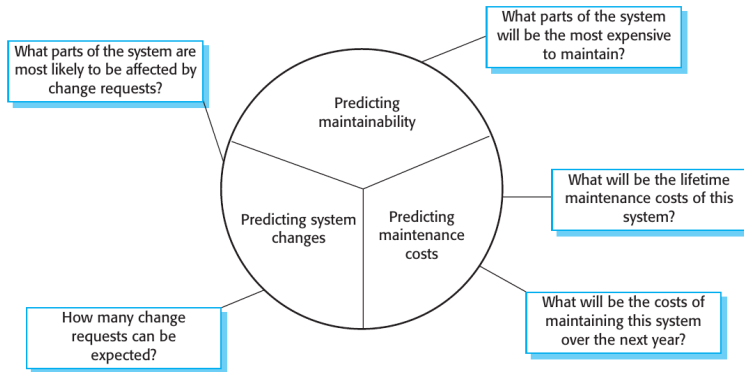
- Di solito sono superiori ai costi di sviluppo (da 2 a 100 volte a seconda dell'applicazione).
- Influenzato da fattori tecnici e non tecnici.
- Aumenta con la manutenzione del SW. La manutenzione corrompe la struttura del SW rende più difficile un'ulteriore manutenzione.
- I SW obsoleti possono avere alti costi di supporto (ad esempio, vecchi linguaggi, compilatori, ecc.).

Costi di manutenzione

- Di solito è più costoso aggiungere nuove funzionalità a un sistema durante la manutenzione che aggiungere le stesse funzionalità durante lo sviluppo.
 - Un nuovo team deve comprendere i programmi in corso di manutenzione.
 - Separare la manutenzione dallo sviluppo significa che non c'è alcun incentivo per il team di sviluppo a scrivere SW manutenibile.
 - Il lavoro di manutenzione del programma è impopolare
 - Il personale addetto alla manutenzione è spesso inesperto e ha una conoscenza limitata del settore.
 - Quando i programmi invecchiano, la loro struttura si degrada e diventa più difficile modificarli.

Previsione di manutenzione

- La previsione della manutenzione si occupa di valutare quali parti del sistema possono causare problemi e avere costi di manutenzione elevati.
 - L'accettazione delle modifiche dipende dalla manutenibilità dei componenti interessati dalla modifica;
 - L'implementazione delle modifiche degrada il sistema e ne riduce la manutenibilità;
 - I costi di manutenzione dipendono dal numero di modifiche e i costi di modifica dipendono dalla manutenibilità.



Previsione di cambiamento

- La previsione del numero di cambiamenti richiede la comprensione delle relazioni tra un sistema e il suo ambiente.
- I sistemi strettamente accoppiati (tightly coupled, quando una serie di componenti di un'applicazione sono interdipendenti) richiedono modifiche ogni volta che l'ambiente viene modificato.
- I fattori che influenzano questa relazione sono
 - Numero e complessità delle interfacce del sistema;
 - Numero di requisiti di sistema intrinsecamente volatili;
 - I processi aziendali in cui viene utilizzato il sistema.

Metriche di complessità

- Le previsioni sulla manutenibilità possono essere fatte valutando la complessità dei componenti del sistema.
- La maggior parte degli sforzi di manutenzione viene spesa per un numero relativamente piccolo di componenti del sistema.
- La complessità dipende da
 - Complessità delle strutture di controllo;
 - Complessità delle strutture dati;
 - Dimensioni dell'oggetto, del metodo (procedura) e del modulo.

Metriche di processo

- Le metriche di processo possono essere utilizzate per valutare la manutenibilità.
 - Numero di richieste di manutenzione correttiva;
 - Tempo medio richiesto per l'analisi d'impatto;
 - Tempo medio di implementazione di una richiesta di modifica;
- L'aumento di uno o di tutti questi elementi può indicare un calo della manutenibilità.

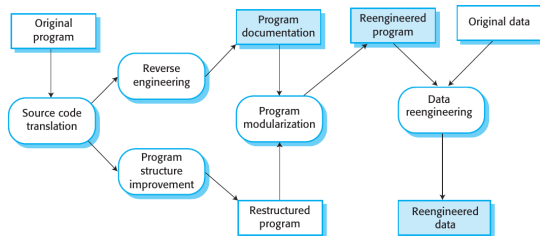
Reingegnerizzazione del SW

- Ristrutturazione o riscrittura parziale o totale di un sistema legacy senza modificarne la funzionalità.
- Applicabile quando alcuni ma non tutti i sottosistemi di un sistema più ampio di un sistema più grande richiedono una manutenzione.
- La reingegnerizzazione comporta l'aggiunta di sforzi per rendere più il sistema più facile da mantenere.
- Il sistema può essere ristrutturato e ridocumentato.

Vantaggi della reingegnerizzazione

- Rischio ridotto
 - Lo sviluppo di nuovo SW comporta un rischio elevato. Ci possono essere problemi di sviluppo, di personale e di specifiche.
- Costo ridotto
 - Il costo della reingegnerizzazione è spesso significativamente inferiore a quello dello sviluppo di un nuovo SW.

Attività di reingegnerizzazione dei processi



- Traduzione del codice sorgente
 - Convertire il codice in un nuovo linguaggio.
- Ingegneria inversa
 - Analizzare il programma per comprenderlo;
- Miglioramento della struttura del programma
 - Ristrutturazione automatica per la comprensibilità;
- Modularizzazione del programma
 - Riorganizzare la struttura del programma;
- Reingegnerizzazione dei dati
 - Pulire e ristrutturare i dati del sistema.

Fattori di costo della reingegnerizzazione

- La qualità del SW da reingegnerizzare.
- Lo strumento di supporto disponibile per la reingegnerizzazione.
- L'entità della conversione dei dati richiesta.
- La disponibilità di personale esperto per la reingegnerizzazione.
 - Questo può essere un problema per i vecchi sistemi basati su tecnologie non più ampiamente utilizzate.

Rifattorizzazione

- Il refactoring è il processo di miglioramento di un programma per rallentare la degradazione attraverso il cambiamento.
- Si può pensare al refactoring come a una "manutenzione preventiva" che riduce i problemi dei cambiamenti futuri.
- Il refactoring consiste nel modificare un programma per migliorarne la struttura, ridurne la complessità o renderlo più comprensibile.
- Quando si rifattorizza un programma, non bisogna aggiungere funzionalità, ma concentrarsi sul miglioramento del programma.

Refactoring e reingegnerizzazione

- La **reingegnerizzazione** avviene dopo che un sistema è stato mantenuto per un certo periodo di tempo e i costi di manutenzione stanno aumentando. Si utilizzano strumenti automatizzati per elaborare e reingegnerizzare un sistema legacy per creare un nuovo sistema più manutenibile.
- Il **refactoring** è un processo di miglioramento continuo durante il processo di sviluppo e di evoluzione. Ha lo scopo di evitare il degrado della struttura e del codice che aumenta i costi e le difficoltà di manutenzione di un sistema.

“Cattivi odori” nel codice del programma

- Codice duplicato

- Lo stesso codice o un codice molto simile può essere incluso in punti diversi di un programma. Questo può essere rimosso e implementato come un singolo metodo o funzione che viene richiamato a seconda delle necessità.

- Metodi lunghi

- Se un metodo è troppo lungo, deve essere riprogettato come una serie di metodi più brevi.

- Dichiarazioni di tipo "switch" (case)

- Spesso si tratta di duplicazioni, in cui lo switch dipende dal tipo di valore. Le istruzioni switch possono essere sparse per il programma. Nei linguaggi orientati agli oggetti, è spesso possibile utilizzare il polimorfismo per ottenere lo stesso risultato.

- Raggruppamento dei dati

- Gli ammassi di dati si verificano quando lo stesso gruppo di dati (campi nelle classi, parametri nei metodi) si ripresenta in più punti di un programma. Spesso è possibile sostituirli con un oggetto che incapsuli tutti i dati.